

Experiment No. 5

Feature Extraction and Image Analysis using SIFT and HOG Descriptors

Aim: To implement Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG) techniques for feature extraction and analyze their effectiveness in image representation, object recognition, image matching, and computer vision applications.

Theory

Feature extraction converts raw image pixels into meaningful information called **features**. These features can be used for image matching, object recognition, tracking, classification, and detection.

SIFT

SIFT (Scale-Invariant Feature Transform) detects distinctive keypoints and generates descriptors around them. It is designed to be robust to changes in **scale and rotation** and reasonably robust to illumination and viewpoint changes.

HOG

Histogram of Oriented Gradients (HOG) describes an image using the distribution of local edge/gradient directions. The image is divided into small cells, gradient orientations are accumulated into histograms, and neighboring cells are grouped into blocks for normalization.

Main Difference

- **SIFT:** Local, keypoint-based descriptor; excellent for image matching and feature correspondence.
- **HOG:** Dense gradient-based descriptor; especially useful for shape and object detection tasks.

```
# Step 1: Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from google.colab import files

print("Libraries imported successfully.")

Libraries imported successfully.

# Step 2: Upload and load an image
uploaded = files.upload()
```

```
file_name = next(iter(uploaded))
img = cv2.imread(file_name)

if img is None:
    raise ValueError("Image could not be loaded. Please upload a valid image.")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

print("Image:", file_name)
print("Image size:", img.shape)

plt.figure(figsize=(7, 5))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.show()

<IPython.core.display.HTML object>

Saving 9011064.webp to 9011064.webp
Image: 9011064.webp
Image size: (2160, 3840, 3)
```

Original Image



Step 3: Basic Preprocessing

The image is converted to grayscale because both SIFT visualization and HOG extraction can be performed using intensity information. A small amount of Gaussian smoothing can optionally reduce insignificant noise.

```
# Step 3: Basic preprocessing
gray_blur = cv2.GaussianBlur(gray, (3, 3), 0)

plt.figure(figsize=(7, 5))
plt.imshow(gray_blur, cmap="gray")
plt.title("Preprocessed Grayscale Image")
plt.axis("off")
plt.show()
```

Preprocessed Grayscale Image



Step 4: SIFT Feature Extraction

SIFT detects important local keypoints and computes a descriptor for each keypoint.

The descriptor is a 128-dimensional vector for each detected SIFT keypoint.

```
# Step 4: Create SIFT detector and extract features
sift = cv2.SIFT_create()

keypoints, sift_descriptors = sift.detectAndCompute(gray_blur, None)

print("Number of SIFT keypoints:", len(keypoints))

if sift_descriptors is not None:
```

```

    print("SIFT descriptor shape:", sift_descriptors.shape)
else:
    print("No SIFT descriptors were detected.")

Number of SIFT keypoints: 1288
SIFT descriptor shape: (1288, 128)

# Step 5: Visualize SIFT keypoints
sift_image = cv2.drawKeypoints(
    img,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(9, 6))
plt.imshow(cv2.cvtColor(sift_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()

```

SIFT Keypoints



Step 6: HOG Feature Extraction

HOG uses image gradients to describe local shape information.

Parameters used:

- Orientations = 9
- Pixels per cell = (8, 8)
- Cells per block = (2, 2)
- Block normalization = L2-Hys

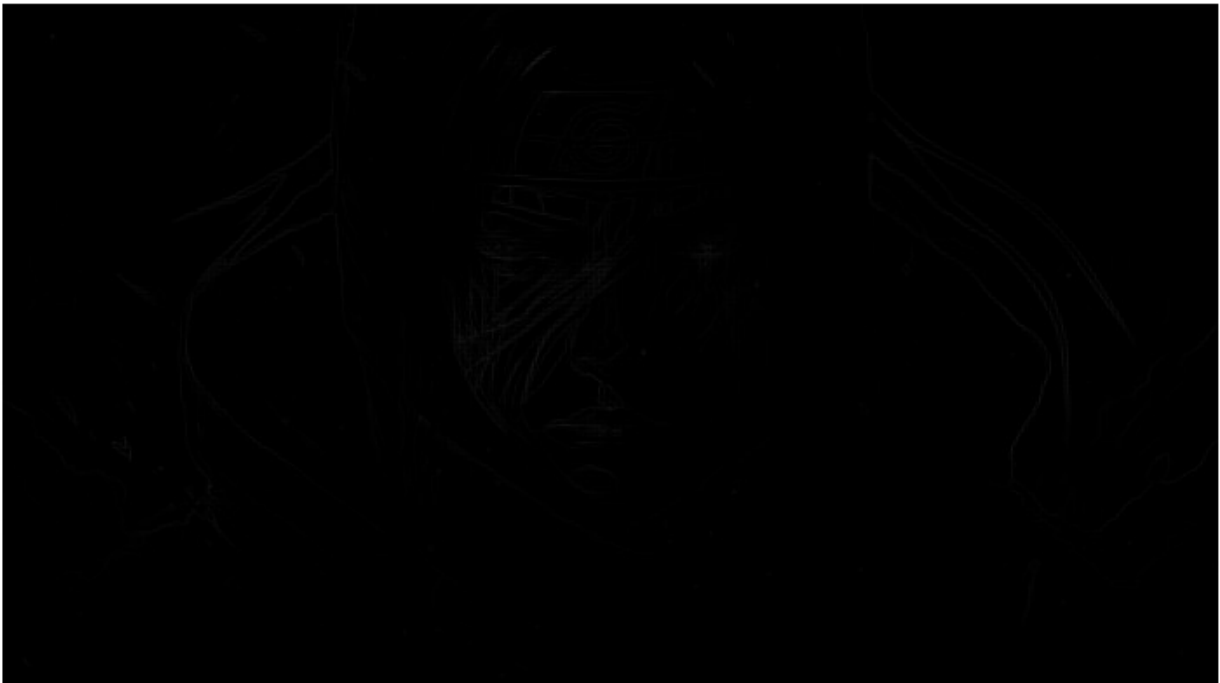
```
# Step 6: Extract HOG features
hog_features, hog_image = hog(
    gray_blur,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    block_norm="L2-Hys",
    visualize=True,
    feature_vector=True
)

print("Number of HOG features:", len(hog_features))

plt.figure(figsize=(9, 6))
plt.imshow(hog_image, cmap="gray")
plt.title("HOG Visualization")
plt.axis("off")
plt.show()
```

Number of HOG features: 4638636

HOG Visualization



Step 7: SIFT vs HOG Comparison

Property	SIFT	HOG
Feature type	Local keypoint-based	Dense gradient-based
Main information	Local texture/structure	Shape and edge distribution
Scale invariance	Yes	Not inherently scale invariant
Rotation invariance	Yes	Limited/not inherently rotation invariant
Descriptor	Local descriptor around keypoints	Histograms of gradient orientations
Best suited for	Matching, recognition, tracking	Object detection and shape analysis
Computational cost	Relatively higher per detected feature	Efficient for fixed-size image windows

Step 8: Basic Image Matching using SIFT

To demonstrate SIFT matching, upload a second image that contains the same object/scene from a slightly different viewpoint, scale, or rotation.

The code below performs descriptor matching using a **FLANN-based matcher** and Lowe's ratio test.

```
# Step 8: Upload a second similar image for SIFT matching
print("Upload a second image containing the same or similar
object/scene.")
uploaded2 = files.upload()

file_name2 = next(iter(uploaded2))
img2 = cv2.imread(file_name2)

if img2 is None:
    raise ValueError("Second image could not be loaded.")

gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
gray2 = cv2.GaussianBlur(gray2, (3, 3), 0)

keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

print("Keypoints in first image:", len(keypoints))
print("Keypoints in second image:", len(keypoints2))

if sift_descriptors is None or descriptors2 is None:
    print("Matching cannot be performed because one image has no
descriptors.")
else:
    # FLANN parameters for SIFT's floating-point descriptors
    index_params = dict(algorithm=1, trees=5)
```

```

search_params = dict(checks=50)

flann = cv2.FlannBasedMatcher(index_params, search_params)

matches = flann.knnMatch(
    sift_descriptors.astype(np.float32),
    descriptors2.astype(np.float32),
    k=2
)

good_matches = []

for pair in matches:
    if len(pair) == 2:
        m, n = pair
        if m.distance < 0.7 * n.distance:
            good_matches.append(m)

print("Good matches:", len(good_matches))

matched_image = cv2.drawMatches(
    img,
    keypoints,
    img2,
    keypoints2,
    good_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16, 7))
plt.imshow(cv2.cvtColor(matched_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Feature Matching")
plt.axis("off")
plt.show()

```

Upload a second image containing the same or similar object/scene.

<IPython.core.display.HTML object>

Saving 9011064.webp to 9011064 (1).webp

Keypoints in first image: 1288

Keypoints in second image: 1288

Good matches: 1288

SIFT Feature Matching



Observations

1. SIFT detects distinctive local keypoints in textured and structurally important image regions.
2. SIFT descriptors are useful for matching the same object or scene across changes in scale and rotation.
3. HOG captures the distribution of gradient directions and represents the shape and edge structure of an image.
4. HOG is especially useful when object shape and contours are important.
5. SIFT is more suitable for local feature correspondence, while HOG is commonly used with object-detection/classification pipelines.
6. SIFT may detect a different number of keypoints depending on image texture, while HOG produces a fixed-size feature vector for a fixed image size and parameter configuration.
7. The quality of both descriptors depends on image resolution, preprocessing, parameter selection, and the application.

Questions and Answers

1. What is feature extraction, and why is it important in computer vision?

Answer: Feature extraction is the process of converting raw image data into meaningful and compact representations such as edges, keypoints, textures, shapes, or descriptors. It reduces irrelevant information and provides useful input for image matching, classification, recognition, and object detection.

2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).

Answer: SIFT detects stable keypoints at different scales using scale-space analysis. It assigns an orientation to each keypoint and constructs a local descriptor from gradient information around it. This makes the descriptor robust to scale and rotation changes and useful for matching corresponding features between images.

3. What are keypoints and feature descriptors in image analysis?

Answer: **Keypoints** are distinctive image locations such as corners, blobs, or textured regions that can be reliably detected. A **feature descriptor** is a numerical representation of the local region around a keypoint, used to compare and match features between images.

4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.

Answer: HOG divides an image into small cells, calculates image gradients, and creates histograms showing the distribution of gradient orientations. Histograms are normalized over blocks to reduce illumination effects. HOG is significant because it effectively represents object shape, contours, and edge structure.

5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

Answer: SIFT is a local feature descriptor that is robust to scale and rotation and is well suited for image matching, tracking, and recognition. HOG is a dense gradient descriptor that is efficient for fixed-size detection windows and is well suited for shape-based object detection. SIFT can require more computation when many keypoints are detected, while HOG's cost depends strongly on image size and descriptor parameters.

6. Why is SIFT considered invariant to scale and rotation?

Answer: SIFT searches for keypoints across multiple image scales and assigns a dominant orientation to each keypoint. The descriptor is then constructed relative to that scale and orientation, making the resulting feature representation largely independent of changes in image size and rotation.

7. Mention three real-world applications where HOG descriptors are commonly used.

Answer:

1. Pedestrian detection.
2. Vehicle and object detection.
3. Human/object shape analysis in surveillance and computer vision systems.

8. Why is feature extraction performed before image classification or object detection?

Answer: Raw images contain a large amount of pixel information. Feature extraction converts this information into meaningful representations such as shape, texture, edges, and local structures. These representations can make classification and detection more efficient and easier for a model or algorithm.

9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

Answer: Handcrafted descriptors such as SIFT and HOG are interpretable, relatively lightweight, and can work well with limited training data. However, their performance depends heavily on manually selected features and parameters. Deep learning can automatically learn complex features from data and often performs better on large datasets, but it generally requires more computational resources and training data.

10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

Answer: Feature extraction provides distinctive numerical representations that can be compared between images. SIFT can find corresponding local features for image matching and recognition, while HOG can represent shape and edge patterns for object detection. These descriptors therefore provide useful information for identifying objects, matching images, and recognizing visual patterns.

Result

SIFT and HOG feature extraction techniques were successfully implemented. SIFT detected local keypoints and generated descriptors suitable for feature matching, while HOG generated gradient-based features representing image shape and edge information. The experiment demonstrated that SIFT is highly useful for local feature matching and recognition, whereas HOG is particularly effective for shape-based object detection.

Conclusion

The experiment successfully demonstrated two important handcrafted feature extraction techniques used in computer vision. **SIFT** provides distinctive local descriptors with strong scale and rotation robustness, making it suitable for image matching and recognition. **HOG** represents local gradient orientation patterns and is highly useful for shape-based object detection. The choice between SIFT and HOG depends on the requirements of the particular computer vision application.